

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Kelvin Krismora Gultom - 5024231062

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler merupakan komponen utama pada sistem tertanam (*embedded system*) yang digunakan secara luas pada perangkat elektronik, otomasi industri, robotika, hingga *Internet of Things* (IoT). Penguasaan dasar pemrograman mikrokontroler menjadi prasyarat sebelum mempelajari topik lanjutan seperti akuisisi sensor, komunikasi data, dan sistem waktu nyata.

Praktikum Modul 1 memperkenalkan dua platform mikrokontroler yang umum digunakan, yaitu STM32 Blackpill dan ESP32, beserta lingkungan pengembangan PlatformIO pada Visual Studio Code. Kedua platform tersebut memiliki karakteristik yang berbeda, baik dari sisi arsitektur, mekanisme pemrograman (*flashing*), maupun *framework* yang tersedia. Pemahaman terhadap perbedaan tersebut diperlukan agar pemilihan platform dan *framework* dapat disesuaikan dengan kebutuhan aplikasi.

Selain itu, praktikum ini membahas tiga mode dasar akses GPIO, yaitu *digital output*, *digital input*, dan *analog input*. Pada *digital input*, pembacaan tombol memerlukan konfigurasi *pull-up* atau *pull-down* agar pin tidak dalam kondisi *floating*, serta penanganan *bouncing* agar satu aksi penekanan tidak terbaca sebagai beberapa kejadian. Pada *analog input*, ADC memungkinkan pembacaan besaran fisis yang kontinu, seperti intensitas cahaya melalui LDR. Ketiga konsep ini merupakan fondasi bagi hampir seluruh aplikasi antarmuka mikrokontroler dengan dunia fisik.

1.2 Dasar Teori

1.2.1 PlatformIO dan Visual Studio Code

PlatformIO adalah ekosistem pengembangan *embedded* lintas platform yang mendukung ratusan *board* dan berbagai *framework* (Arduino, STM32Cube/HAL, ESP-IDF, dan lainnya) dalam satu antarmuka yang seragam. *Framework* yang berbeda dapat dijalankan pada chip yang sama hanya dengan mengubah konfigurasi pada berkas `platformio.ini`. Struktur proyek PlatformIO terdiri atas `platformio.ini` (konfigurasi *board*, *framework*, dan protokol *upload*), `src/` (kode program utama), `include/` (berkas *header*), dan `lib/` (pustaka tambahan).

1.2.2 STM32 Blackpill dan Framework Arduino

STM32 Blackpill adalah *development board* berbasis mikrokontroler STM32F401CC (ARM Cortex-M4 32-bit dengan FPU) dengan *clock* hingga 84 MHz, memori *flash* 256 KB, dan RAM 64 KB. LED *onboard* terhubung pada pin PC13 dengan logika aktif LOW, sedangkan tombol pengguna terhubung pada pin PA0. *Board* ini tidak memiliki *programmer onboard*, sehingga pemrograman dilakukan menggunakan ST-Link eksternal melalui protokol SWD (SWDIO, SWCLK, GND, dan 3,3V).

Framework Arduino untuk STM32 (STM32duino) menyediakan fungsi yang umum dikenal seperti `pinMode()`, `digitalWrite()`, dan `delay()`, serta mendukung penamaan pin sesuai label port fisik (misalnya PC13 dan PA0). Di balik abstraksi tersebut, *framework* ini tetap dibangun di atas HAL (*Hardware Abstraction Layer*) milik STMicroelectronics.

1.2.3 ESP32 dan Framework ESP-IDF

ESP32 adalah mikrokontroler 32-bit *dual-core* (Xtensa LX6) dengan WiFi dan Bluetooth terintegrasi. *Board* ESP32 DevKit dilengkapi chip USB-to-Serial (misalnya CP2102 atau CH340) sehingga dapat

diprogram langsung melalui kabel USB.

ESP-IDF (*Espressif IoT Development Framework*) adalah *framework* resmi dari Espressif yang dibangun di atas FreeRTOS. Titik masuk program berupa fungsi `app_main()` yang berjalan sebagai *task* FreeRTOS, sedangkan pada *framework* Arduino titik masuknya berupa fungsi `setup()` dan `loop()`. Penundaan waktu pada Arduino menggunakan `delay(ms)`, sedangkan pada ESP-IDF menggunakan `vTaskDelay(pdMS_TO_TICKS(ms))`. Tingkat abstraksi Arduino tergolong tinggi sehingga mudah dipelajari, sedangkan ESP-IDF berada pada tingkat rendah hingga menengah sehingga lebih *native* dan lebih kuat. Fitur RTOS pada Arduino tersembunyi atau terbatas, sedangkan pada ESP-IDF dapat diakses langsung berupa *task*, *semaphore*, dan *queue*.

1.2.4 Konfigurasi Pull-up dan Pull-down Resistor

Pin GPIO yang tidak terhubung ke level tegangan tertentu berada dalam kondisi *floating*, sehingga nilai bacaannya tidak pasti akibat *noise*. Kondisi ini dicegah dengan resistor *pull-up* atau *pull-down*. Pada konfigurasi *pull-up*, pin ditarik ke VCC (3,3V) melalui resistor sehingga terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan (tombol menghubungkan pin ke GND). Pada konfigurasi *pull-down*, pin ditarik ke GND sehingga terbaca LOW saat tombol tidak ditekan dan HIGH saat ditekan (tombol menghubungkan pin ke VCC).

Resistor tersebut dapat dipasang secara eksternal (umumnya 10 kΩ) maupun diaktifkan secara internal melalui *firmware*, yaitu `INPUT_PULLUP` dan `INPUT_PULLDOWN` pada *framework* Arduino untuk ESP32. Pin *input-only* (GPIO 34–39) tidak memiliki resistor *pull* internal sehingga wajib menggunakan resistor eksternal.

1.2.5 Debouncing

Kontak mekanik pada tombol menghasilkan beberapa transisi HIGH–LOW dalam waktu sangat singkat akibat getaran fisik saat kontak bersentuhan atau terlepas, yang disebut *bouncing*. Tanpa penanganan, satu kali penekanan dapat terbaca sebagai beberapa kejadian. *Debouncing* memastikan hanya satu transisi valid yang terdeteksi dengan menunggu sinyal stabil selama periode tertentu (misalnya 20–50 ms) sebelum dianggap sah.

1.2.6 ADC sebagai Mode Akses GPIO Ketiga

Selain *digital output* dan *digital input*, GPIO dapat berfungsi sebagai *analog input* melalui ADC (*Analog-to-Digital Converter*) yang mengubah tegangan kontinu menjadi nilai digital diskrit. ESP32 memiliki ADC internal dengan resolusi bawaan 12-bit (nilai 0–4095) dan rentang tegangan sekitar 0–3,3V, yang diakses melalui `analogRead(pin)`. Pin ADC1 (misalnya GPIO 32–39) lebih disarankan dibandingkan ADC2 karena ADC2 memiliki keterbatasan saat modul WiFi aktif. Pada percobaan, LDR dirangkai sebagai pembagi tegangan bersama resistor 10 kΩ sehingga perubahan intensitas cahaya menghasilkan perubahan tegangan pada pin ADC.

2 Tugas Pendahuluan

1. **Perbedaan framework Arduino dan ESP-IDF**

Arduino merupakan *framework* berabstraksi tinggi yang menyederhanakan program menjadi struktur `setup()` dan `loop()` dengan fungsi-fungsi siap pakai, sedangkan ESP-IDF merupakan *framework* resmi Espressif yang bersifat *native*, dibangun di atas FreeRTOS, dengan titik masuk `app_main()` dan struktur berbasis *task* dan *component*. Pada ESP32, *framework* Arduino sendiri dibangun di atas ESP-IDF sehingga ESP-IDF berada pada lapisan yang lebih rendah.

Kelebihan Arduino adalah kurva belajar yang landai, sintaks sederhana, pustaka komunitas yang sangat banyak, serta prototipe yang dapat dibuat dengan cepat. Kekurangannya adalah fitur RTOS yang tersembunyi atau terbatas, kendali tingkat rendah yang terbatas, dan adanya *overhead* dari lapisan abstraksi.

Kelebihan ESP-IDF adalah akses penuh ke fitur ESP32 (konfigurasi WiFi tingkat rendah, penjadwalan *task*, *semaphore*, dan *queue*), kendali *hardware* yang lebih detail, serta kesesuaian untuk produk IoT tingkat lanjut. Kekurangannya adalah kurva belajar yang curam, kode yang lebih panjang, dan konfigurasi proyek yang lebih kompleks.

2. **Alasan STM32 Blackpill memerlukan ST-Link, sedangkan ESP32 dapat diprogram melalui USB**

   Proses *flashing* memerlukan jalur komunikasi antara komputer dan mikrokontroler. Pada STM32 Blackpill tidak terdapat chip USB-to-Serial maupun *programmer onboard*, sehingga komputer tidak memiliki jembatan komunikasi bawaan menuju pin UART atau SWD mikrokontroler. Oleh karena itu, ST-Link eksternal diperlukan sebagai *programmer*/*debugger* yang menghubungkan komputer (USB) dengan mikrokontroler melalui protokol SWD (SWDIO, SWCLK, GND, 3,3V).

   Pada ESP32 DevKit terdapat chip USB-to-Serial (CP2102/CH340) yang mengubah sinyal USB dari komputer menjadi sinyal serial UART yang dipahami ESP32. *Bootloader* bawaan ESP32 menerima program melalui UART tersebut, dan masuk ke mode *download* secara otomatis melalui pengaturan pin EN dan GPIO0 oleh chip USB-to-Serial. Dengan demikian, ESP32 dapat diprogram langsung melalui kabel USB tanpa *programmer* eksternal. Chip USB-to-Serial inilah yang berperan sebagai pengganti fungsi *programmer* pada ESP32.

3. **Perbedaan kondisi logika pull-up dan pull-down**

   Pada konfigurasi *pull-up*, pin ditarik ke VCC melalui resistor sehingga saat tombol tidak ditekan pin terbaca HIGH. Saat tombol ditekan, pin terhubung ke GND sehingga terbaca LOW.

   Pada konfigurasi *pull-down*, pin ditarik ke GND melalui resistor sehingga saat tombol tidak ditekan pin terbaca LOW. Saat tombol ditekan, pin terhubung ke VCC sehingga terbaca HIGH.

   Dengan demikian, logika kedua konfigurasi saling berkebalikan: *pull-up* bernilai HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan *pull-down* bernilai LOW saat tidak ditekan dan HIGH saat ditekan.

4. **Fenomena bouncing pada tombol mekanik**

   Bouncing terjadi karena kontak logam pada tombol bersifat elastis dan tidak langsung diam saat bersentuhan atau terlepas. Kontak memantul beberapa kali dalam rentang waktu singkat (umumnya beberapa milidetik) sehingga sinyal berubah HIGH–LOW berulang kali sebelum stabil. Mikrokontroler yang membaca sinyal dengan cepat akan menganggap setiap transisi sebagai penekanan terpisah, sehingga satu aksi penekanan dapat terbaca beberapa kali. Dua pendekatan untuk mengatasinya adalah sebagai berikut.

(a) *Debouncing* perangkat lunak: sinyal dianggap valid hanya jika nilainya stabil selama periode tertentu (misalnya 20–50 ms) menggunakan penghitung waktu seperti `millis()`, seperti pada Percobaan 4.

(b) *Debouncing* perangkat keras: pemasangan filter RC (resistor dan kapasitor) pada tombol untuk meredam perubahan sinyal cepat, dapat dikombinasikan dengan *Schmitt trigger*, atau menggunakan rangkaian *latch* SR.

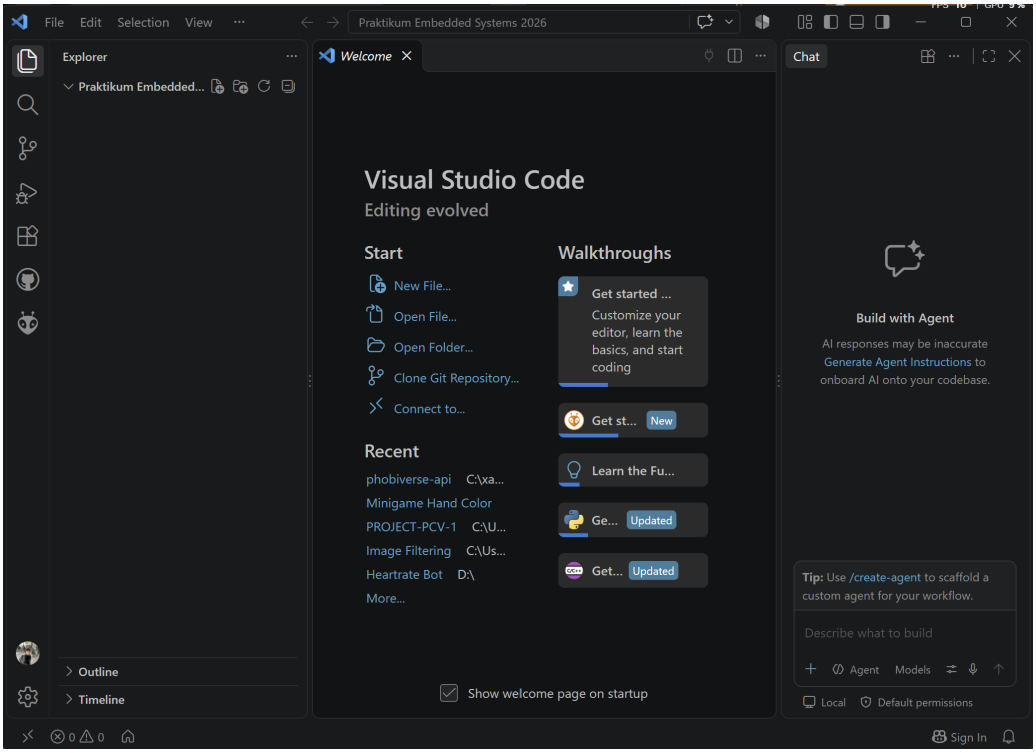
5. **Cara kerja ADC dan pengaruh resolusi bit**

   ADC mengubah sinyal analog menjadi nilai digital melalui tiga tahap, yaitu *sampling* (pencuplikan tegangan pada waktu tertentu), *quantization* (pemetaan tegangan ke salah satu dari sejumlah level diskrit), dan *encoding* (representasi level tersebut dalam bilangan biner). Nilai digital diperoleh dari perbandingan tegangan masukan terhadap tegangan referensi, kemudian dikalikan dengan jumlah level maksimum pada resolusi tersebut. Sebagai contoh, pada ADC 12-bit dengan tegangan referensi 3,3 V, tegangan masukan 1,65 V (setengah dari tegangan referensi) menghasilkan nilai digital sekitar 2048.

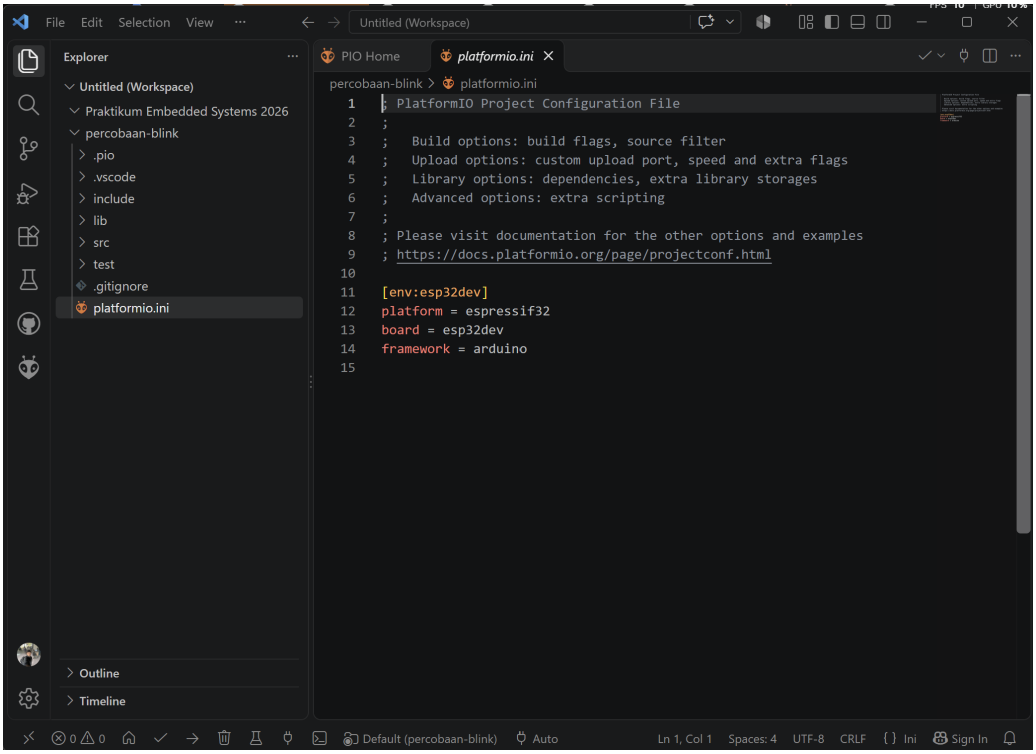
   Semakin besar resolusi bit, semakin banyak level diskrit yang tersedia sehingga langkah kuantisasi semakin kecil dan hasil pembacaan semakin presisi. Dengan tegangan referensi 3,3 V, ADC 8-bit memiliki 256 level (0–255) dengan ukuran satu langkah sekitar 12,94 mV. ADC 10-bit memiliki 1024 level (0–1023) dengan ukuran satu langkah sekitar 3,23 mV. ADC 12-bit memiliki 4096 level (0–4095) dengan ukuran satu langkah sekitar 0,81 mV. Artinya, perubahan tegangan yang lebih kecil masih dapat dibedakan pada resolusi yang lebih tinggi.

6. **Set-up tools: VSCode, PlatformIO (ESP-IDF dan STM32), dan Serial Plotter**

   Set-up perangkat lunak telah dilakukan dengan memasang Visual Studio Code, ekstensi PlatformIO IDE, serta ekstensi *serial plotter*. Platform `espressif32` dengan *framework* ESP-IDF dan platform `ststm32` telah terpasang melalui PlatformIO. Bukti keberhasilan set-up ditunjukkan pada Gambar 1–**??**.



Gambar 1: Visual Studio Code dengan ekstensi PlatformIO yang telah terpasang



Gambar 2: Platform `espressif32` (ESP-IDF) dan `ststm32` yang telah terpasang pada PlatformIO